

Cassandra's architecture and its components

Cassandra is based on the ring architecture. This is a collection of nodes that are interconnected with each other. All nodes are considered as equal and have a similar role. There are no slave or master nodes.

The Cassandra architecture comprises many components. These are:

- Nodes
- Data centre
- Clusters
- Commit logs
- Memtables
- SSTables
- Bloom filters

Let us explore each component to understand the architecture better.

Node: A node is where data is stored. We can perform read and write operations on any node. If, at any time, a node goes down, then read and write operations or requests can be performed by other nodes without any failure.

Data centre: This is a collection of associated nodes, and consists of the same data across all nodes.

Cluster: This comprises either a single or multiple data centres.

Commit log: For durability purposes and to avoid losing data, Cassandra appends writes to the commit log.

Memtable: This is an in-memory data structure that Cassandra stores the data in. After performing the commit log operation, data is written to the memtable.

SSTable: This is a disk file; once the memtable is full or exceeds the configurable threshold value (while configuring or installing Cassandra, this value is set), then data will be flushed from the memtable and stored in the SSTable.

Bloom filter: This is used to find an appropriate SSTable that holds the required data.

Read and write operations

Unlike some conventional relational databases, Cassandra can perform read and write operations to any node. Here,

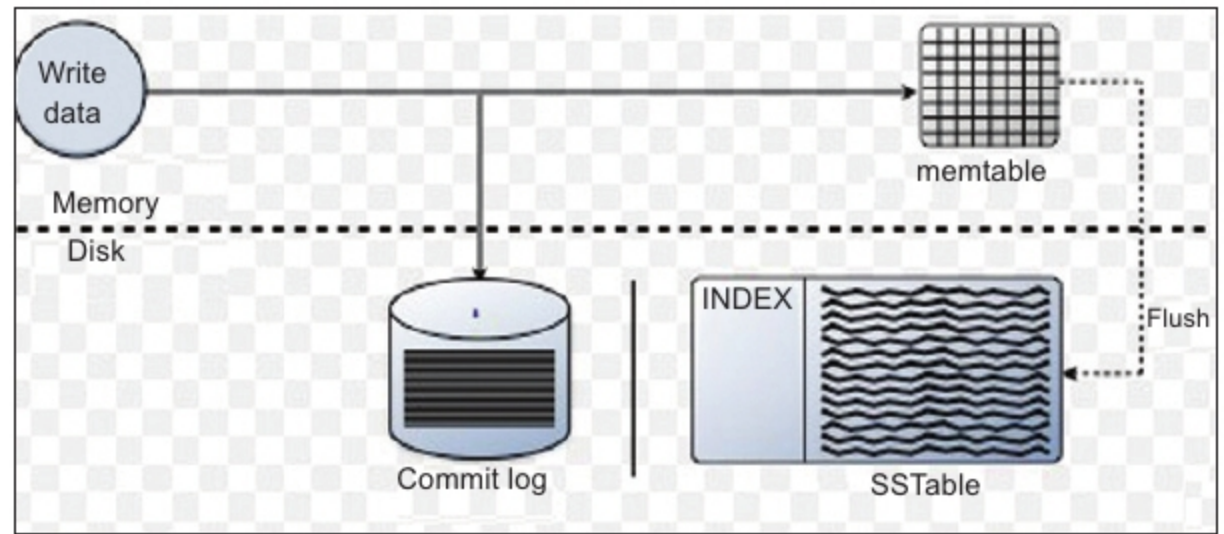


Figure 3: The write operation process

Command	Short description
<code>CREATE KEYSPACE < identifier > WITH < properties ></code>	This is used to create the keyspace.
<code>Describe keyspaces</code>	This command gives the list of all keyspaces that are present.
<code>ALTER KEYSPACE < identifier > WITH < properties ></code>	It is used to alter the identifier and keyspaces' properties.
<code>Drop Keyspace <keyspace_name></code>	Used to drop the keyspace.

we will discuss the steps to perform a write and read operation.

The write operation

- a. Each write operation of the nodes is picked up by the commit logs written in the nodes.
- b. After that, this data is captured and stored in the memtable. If the memtable is full, data will be written to the SSTable data file.
- c. As shown in Figure 3, once the memtable is full, the flush process comes into play. This is the process of turning the memtable into an SSTable.
- d. Cassandra automatically discards unnecessary data from SSTable.

The read operation

Once the write operation is done, Cassandra gets the value from the

memtable and inspects the bloom filter to find the right SSTable that holds the necessary data, which it will start to read.

Understanding basic Cassandra query language (CQL) commands

Let us now explore the basic Cassandra query language (CQL) syntax. But before that, we should understand what CQL is. Basically, Cassandra query language (CQL) is for the Cassandra (C*) database. It is very similar to the structured query language (SQL). The way to interact with Cassandra is through the CQL shell/ cqlsh. In cqlsh, we can perform many operations like create, alter and drop keyspace, as shown in the table above.

In the following section, we will go through each command with an example.

```
KJagtap@cqlsh> CREATE KEYSPACE cassandradb WITH replication = ('class': 'NetworkTopologyStrategy', 'Analytics': '3') AND durable_writes = true;
KJagtap@cqlsh> desc keyspace cassandradb ;

CREATE KEYSPACE cassandradb WITH replication = ('class': 'NetworkTopologyStrategy', 'Analytics': '3') AND durable_writes = true;

KJagtap@cqlsh> █
```

Figure 4: Create keyspace command